

Extractive Question Answering With Lightweight Models

Sherouk Rashed
Department of Computer Science
Queen's University
25rnhf@queensu.ca

Mostafa Elsayed
Department of Computer Science
Queen's University
25llky@queensu.ca

Salma Hamad
Department of Computer Science
Queen's University
email3@university.edu

Abstract—This project investigates extractive question answering on the SQuAD v1.1 dataset through a complete pipeline comprising data preprocessing, model optimization, and ablation analysis. We fine-tune DistilBERT and TinyBERT on a 50,000-example subset, achieving a best validation baseline of 67.40% Exact Match (EM) and 78.42% F1. A structured three-stage hyperparameter optimization establishes a stable starting point before two main ablation directions are explored. The layer-freezing ablation shows that freezing the embeddings and the bottom two DistilBERT layers retains 99.3% of full fine-tuning F1 (72.3 vs. 72.8) while reducing trainable parameters by 57.3% and training time by 21%. The data-size ablation reveals diminishing returns beyond 25% of the training data, with an EM gain of only 2.5 points from 25% to 35% despite a 40% increase in training examples. A sequence-length ablation further demonstrates that DistilBERT maintains stable performance across context lengths, while TinyBERT degrades from 34.20% to 27.40% EM as sequence length increases from 128 to 384 tokens. On the held-out test set, DistilBERT achieves 66.80% EM and 77.93% F1, compared with 27.40% EM and 36.02% F1 for TinyBERT. The complete code and reproducibility scripts are available at: <https://github.com/Salma-21/Lightweight-Extractive-Question-Answering-on-SQuAD-Using-DistilBERT>.

Index Terms—Extractive question answering, SQuAD, DistilBERT, TinyBERT, layer freezing, data efficiency, ablation study

I. INTRODUCTION

Extractive question answering (QA) is a fundamental NLP task that aims to return a direct answer to a user's question from a given text passage. Rather than generating an answer, the model selects the most relevant span from the provided context. This makes the task well-suited for applications such as search engines, digital assistants, and educational platforms, where precise and reliable answers are required.

Large transformer models such as BERT [3] have demonstrated strong performance on QA benchmarks, but they require high computational resources for both training and inference. This creates a practical barrier for deployment on resource-constrained hardware. Lightweight transformer models such as DistilBERT [2] and TinyBERT [10] offer promising alternatives, as they reduce model size while preserving much of the language understanding ability of their larger counterparts.

This project focuses on efficient extractive QA on SQuAD v1.1 [1] using lightweight transformer models under CPU-compatible resource constraints. The pipeline proceeds as

follows: first, the SQuAD dataset is prepared and split into training, validation, and test sets; second, the text is preprocessed using transformer tokenization with a sliding-window strategy to handle long contexts while preserving answer spans; third, two baselines are implemented to establish a lower-bound comparison; fourth, DistilBERT and TinyBERT are fine-tuned for span prediction; and finally, optimization and ablation experiments are conducted to study the trade-off between performance and efficiency.

The ablation analysis covers three dimensions: (1) layer freezing, to evaluate whether fixing subsets of model parameters during fine-tuning can reduce training cost without significantly harming QA quality; (2) training data size, to assess how performance scales with the volume of training examples; and (3) sequence length, to characterize the effect of context window size on both accuracy and latency. The overall goal is to provide a practical and reproducible analysis of lightweight transformer models for extractive QA under constrained computational settings.

II. RELATED WORK

A. BERT for Extractive Question Answering

Devlin et al. [3] introduced BERT, a deep bidirectional transformer pre-trained on large text corpora using masked language modeling and next-sentence prediction. For extractive QA, BERT encodes the question and context jointly and predicts the start and end token positions of the answer span. BERT-base achieves 80.8% EM and 88.5% F1 on SQuAD v1.1, establishing a strong performance ceiling for the task. However, its 110 million parameters and multi-layer attention mechanism make it computationally expensive for resource-constrained deployment, motivating the use of distilled alternatives in this project.

B. DistilBERT for Resource-Efficient QA

DistilBERT [2] is a distilled version of BERT designed to reduce model size and inference cost while preserving much of BERT's language understanding ability. The original paper reports that DistilBERT retains 97% of BERT's performance on the GLUE benchmark while being 40% smaller and 60% faster at inference [2]. Because our project focuses on improving efficiency without sacrificing answer quality, DistilBERT

serves as the primary model for optimization and ablation experiments.

C. TinyBERT for Compact Inference

Jiao et al. [10] introduced TinyBERT, a two-stage knowledge distillation framework that transfers knowledge from a BERT teacher at both the pre-training and task-specific stages by distilling attention matrices, hidden states, and output logits. TinyBERT4 achieves 96.8% of BERT-base’s GLUE performance while being $7.5\times$ smaller and $9.4\times$ faster at inference, making it well-suited for latency-sensitive deployments. In this project, TinyBERT serves as a compact comparison model against DistilBERT.

D. Preprocessing, Tokenization, and Chunking

Extractive QA models require careful preprocessing because the answer span must remain aligned with the original context [1]. Aggressive normalization such as stemming or case folding risks distorting gold answer boundaries, so our pipeline applies only light whitespace cleaning and explicit offset validation. The original BERT paper introduced the practice of joint question-context encoding with special separator tokens, and subsequent QA work adopted overlapping stride-based chunking to ensure that answers near chunk boundaries are not discarded during truncation [3], [5]. This directly motivates our use of sliding-window tokenization in the preprocessing pipeline.

E. Layer Freezing in Transformer Fine-Tuning

Layer freezing has been studied as a parameter-efficient fine-tuning strategy for transformer models. The underlying intuition is that lower transformer layers tend to encode general linguistic features, while upper layers adapt more strongly to the target task [4]. Merchant et al. [11] showed that fine-tuning primarily affects the upper layers of BERT, while lower layers preserve general linguistic information. Lee et al. [12] directly studied freezing during fine-tuning and showed that updating only a subset of layers can preserve a large portion of downstream performance. Recent empirical work confirms that selectively freezing the bottom 25–50% of transformer layers can meaningfully cut training cost while retaining task performance [6].

F. Data Efficiency and Scaling

Ablation over training data size is a standard way to study how performance depends on data availability. General findings in NLP show that model performance improves as training data increases, but with diminishing returns beyond a certain point [7]. For extractive QA specifically, prior work has demonstrated that DistilBERT fine-tuned on SQuAD subsets can still produce meaningful results [2], [8]. Fayyaz et al. [13] showed that not all training examples contribute equally during fine-tuning, suggesting that careful data selection can maintain competitive performance with fewer examples.

G. Evaluation of Quality and Efficiency

Recent QA work increasingly reports not only answer quality but also efficiency metrics such as training time and inference latency [8]. A recent comparative study on BERT-family models fine-tuned on SQuAD showed that reporting both EM/F1 and resource metrics gives a more complete picture of model suitability for constrained deployment [9]. Accordingly, our evaluation combines Exact Match and F1 with training time, inference latency, and trainable parameter count across all ablation settings.

III. DATA AND PREPROCESSING

A. Dataset Details

This project uses the Stanford Question Answering Dataset (SQuAD) v1.1 [1]. SQuAD v1.1 is a span-based reading comprehension benchmark built from Wikipedia articles, where each question has a single answerable text span within the context. The dataset was loaded from the official SQuAD files and verified manually to confirm the JSON structure and the correctness of answer offsets. A span alignment audit across all 87,599 examples confirmed zero mismatches. To keep the pipeline feasible under the available compute budget, the training set was limited to 50,000 QA pairs, and the data was split at the article level into 80% train, 10% validation, and 10% test.

B. Preprocessing Steps

The preprocessing was intentionally light because the task is extractive QA, where the answer must remain aligned with its original span in the passage. We therefore avoided aggressive text transformations such as stemming, stopword removal, and case folding, since these operations can break gold answer offsets. Instead, the pipeline applies only minimal normalization: repeated whitespace is collapsed, leading and trailing spaces are stripped, and `answer_start` positions are verified to point to the correct span.

C. Tokenization for DistilBERT

We tokenize the question and context jointly using the DistilBERT tokenizer in the standard QA format, allowing the model to encode both inputs in one sequence with the format `[CLS] question [SEP] context [SEP]`. Because many SQuAD passages exceed the maximum input length, we use a sliding-window strategy with stride 128 so that long contexts can be split into overlapping chunks without losing answers near chunk boundaries. The maximum sequence length is set to 384 tokens, balancing context coverage with memory and training cost. Character-level offset mappings are recorded for converting predicted token positions back to answer strings at inference time. Padding is applied so that all examples in a batch share the same length, as required for efficient batched training.

D. Tokenization for TinyBERT

The TinyBERT tokenizer was used to encode each question–context pair. Truncation was applied only to the context sequence using `truncation="only_second"` to preserve the complete question. A maximum sequence length of 384 tokens and a stride of 128 tokens were used, consistent with the DistilBERT setup. Offset mappings were generated to associate each token with its corresponding character positions, and answer boundaries were converted into token-level positions using these mappings. If a sliding-window segment did not fully contain the answer span, both start and end indices were assigned null labels to prevent incorrect supervision. The generated input IDs, attention masks, and token-level answer boundaries were formatted as PyTorch tensors for model training and validation.

E. Train, Validation, and Test Split

We split the dataset at the article level using an 80/10/10 train/validation/test ratio with a fixed random seed (42). This reduces leakage because questions from the same article remain within the same partition. The validation set is used during optimization to select the best checkpoint, while the test set is reserved exclusively for final evaluation. The same split is reused across all experiments to ensure that comparisons are fair and unaffected by changes in the evaluation data.

IV. METHODOLOGY

A. Models

This project uses DistilBERT [2] as the primary model and TinyBERT [10] as a compact comparison model. Both are distilled versions of BERT designed to be smaller, faster, and less expensive to train while retaining much of BERT’s language understanding ability. Table I summarizes the architectural differences between BERT-base, DistilBERT, and TinyBERT.

TABLE I: Architecture Comparison

	BERT-base	DistilBERT	TinyBERT
Layers	12	6	4
Hidden size	768	768	312
Attention heads	12	12	12
FFN size	3,072	3,072	1,200
Parameters	110M	66M	14.5M
SQuAD F1 ^a	88.5	86.9	87.5 ^b

^a Reported on full SQuAD v1.1 training data.

^b With task-specific distillation fine-tuning.

BERT-base provides a strong representational upper bound at the cost of high computational overhead. DistilBERT reduces depth by half while preserving the same 768 hidden size, retaining most of BERT’s contextual understanding. TinyBERT applies more aggressive compression—four layers, hidden size 312, FFN size 1,200—achieving a $7.8\times$ parameter reduction versus BERT, at the cost of reduced context modelling capacity, as confirmed by our sequence-length ablation.

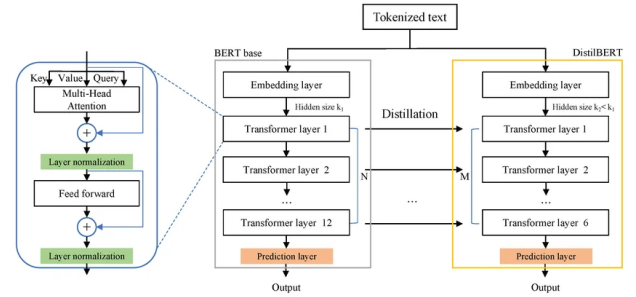


Fig. 1: DistilBERT architecture for extractive QA. Input: [CLS] question [SEP] context [SEP]; Output: start and end logits over all tokens.

B. Baselines

Two baselines are implemented to establish a lower bound for transformer-based span extraction.

BM25 retrieval baseline: Given a question, BM25 ranks passage sentences by term overlap and returns the top-ranked sentence as the predicted answer. BM25 is a classical information retrieval method that does not perform token-level span prediction, and therefore cannot satisfy SQuAD’s exact-match requirement.

TF-IDF cosine similarity baseline: TF-IDF vector representations of each passage sentence are computed, and the sentence with the highest cosine similarity to the question is returned as the predicted answer. Like BM25, this method returns sentence-level candidates rather than precise token spans.

Both baselines confirm that lexical retrieval approaches are fundamentally insufficient for extractive span QA, motivating the use of fine-tuned transformer models.

C. Training Procedure

Both models were fine-tuned on the same 50,000-pair SQuAD v1.1 subset, split 80/10/10 at the article level (seed 42), with `max_length=384` and `stride=128`. Table II summarizes the hyperparameters. DistilBERT converges quickly with a higher learning rate and fewer epochs due to its larger representational capacity. TinyBERT requires a lower learning rate, more epochs, and a warmup schedule to stabilize training given its compressed architecture. Weight decay is applied to both models as a regularization technique to prevent overfitting and improve generalization to unseen data.

D. Baseline Optimization for DistilBERT

Before any ablation, we optimized the DistilBERT baseline in three stages to ensure that ablation results reflect genuine design choices rather than a poorly tuned starting point:

- 1) **Overfitting sanity check:** Trained on 150 samples for 50 epochs to verify that the pipeline, tokenization, and span-label alignment were functioning correctly.
- 2) **Coarse hyperparameter search:** Grid search over learning rate $\in \{1 \times 10^{-5}, 2 \times 10^{-5}, 3 \times 10^{-5}\}$, epochs

TABLE II: Fine-Tuning Hyperparameters

Hyperparameter	DistilBERT	TinyBERT
Learning rate	2×10^{-5}	1×10^{-5}
Effective batch size	32	32
Epochs	3	20
Max seq. length	384	384
Stride	128	128
Warmup ratio	0.1	0.1
Weight decay	0.01	0.01
Early stopping (patience)	2	2
Optimizer	AdamW	AdamW
Seed	42	42

$\in \{2, 3\}$, warmup ratio $\in \{0.0, 0.1\}$, and weight decay $\in \{0.0, 0.01\}$, conducted on 10% of the training data.

- 3) **Fine hyperparameter search:** Refined the learning rate in $[1 \times 10^{-5}, 2 \times 10^{-5}]$ with all other settings fixed at the best coarse configuration.

Table III summarizes the search space and the final configuration selected.

TABLE III: Hyperparameter Search Space and Final Configuration

Parameter	Search Space	Final Value
Learning rate	$1 \times 10^{-5}, 2 \times 10^{-5}, 3 \times 10^{-5}$	2×10^{-5}
Epochs	2, 3	3
Warmup ratio	0.0, 0.1	0.1
Weight decay	0.0, 0.01	0.01
Batch size	Fixed (16)	16
Max seq. length	Fixed (384)	384

E. Layer-Freezing Ablation for DistilBERT

This ablation tests whether DistilBERT can preserve most of its QA performance when a subset of parameters is held fixed during fine-tuning. Lower layers tend to encode general linguistic features, while upper layers are more task-specific and critical for span prediction. Three configurations are compared:

- **full_finetune:** Full fine-tuning of all parameters (baseline).
- **emb_2layers:** Freeze embeddings and the bottom 2 transformer layers.
- **emb_4layers:** Freeze embeddings and the bottom 4 transformer layers (only top 2 layers remain trainable).

F. Data-Size Ablation for DistilBERT

This ablation studies how QA performance scales with training data availability. Three model variants are trained on subsets representing 15%, 25%, and 35% of the 50,000-example training set (approximately 7,510, 12,517, and 17,523 examples). All other settings are fixed at the best configuration from the fine hyperparameter search. Note that while the original plan called for three independent runs per configuration to report mean $\pm$ standard deviation, single runs were conducted due to computational constraints; results should therefore be interpreted as indicative.

G. TinyBERT Layer-Freezing Ablation

Three encoder-freezing configurations are tested for TinyBERT:

- Freeze encoder layer 0 only.
- Freeze encoder layers 0–1.
- Freeze all encoder layers 0–3.

In all configurations, the QA prediction head remains trainable. The best configuration is selected by F1 score, with training time as a secondary criterion.

H. TinyBERT Stratified Data-Size Ablation

Under the best TinyBERT freezing configuration (freeze layer 0), three stratified subsets of the training data are used: 25% (12,631 chunks), 50% (25,263 chunks), and 70% (35,368 chunks). Stratification is performed by SQuAD article title to preserve topic coverage across different subset sizes.

I. Evaluation Metrics

All experiments are evaluated using both quality and efficiency metrics.

Quality Metrics: Exact Match (EM) measures whether the predicted answer exactly matches one of the gold answers after normalization:

$$\text{EM} = \frac{1}{N} \sum_{i=1}^N \mathbf{1}(\hat{a}_i = a_i) \quad (1)$$

where $\hat{a}_i$ is the predicted answer for example i , a_i is the gold answer, and N is the total number of evaluation examples.

F1 score measures token-level overlap between the predicted and gold answers, computed via precision and recall:

$$\text{Precision} = \frac{|\text{Prediction} \cap \text{Gold}|}{|\text{Prediction}|} \quad (2)$$

$$\text{Recall} = \frac{|\text{Prediction} \cap \text{Gold}|}{|\text{Gold}|} \quad (3)$$

$$\text{F1} = \frac{2 \times \text{Precision} \times \text{Recall}}{\text{Precision} + \text{Recall}} \quad (4)$$

EM is a strict binary metric (partial matches score zero), while F1 gives partial credit for overlapping tokens, providing a more nuanced view of answer quality.

Efficiency Metrics: Efficiency is measured via training time (hours), inference latency (ms/sample), and the number of trainable parameters (millions). Reporting these metrics alongside EM and F1 gives a complete picture of the accuracy–efficiency trade-off, which is the primary motivation for using distilled models.

V. EXPERIMENTS AND RESULTS

This section presents results across four stages: baseline evaluation, hyperparameter optimization, DistilBERT ablation studies, and TinyBERT ablation studies. All experiments are evaluated using EM, F1, training time, inference latency, and trainable parameter count as defined in Section IV-I.

A. Baseline Models

1) *Classical Baselines*: Table IV reports the two classical baselines on 500 validation examples. BM25 achieves 0.00% EM and 12.75% F1, while TF-IDF achieves 0.11% EM and 15.25% F1. Both methods return sentence-level candidates rather than token-level spans, making them fundamentally ill-suited for extractive span prediction as measured by SQuAD metrics.

TABLE IV: Classical Baselines on 500 Validation Examples

Model	EM (%)	F1 (%)
BM25 Retrieval	0.00	12.75
TF-IDF Cosine	0.11	15.25

2) *DistilBERT Baseline*: The DistilBERT baseline (distilbert-base-uncased) was fine-tuned for 3 epochs with learning rate 2×10^{-5} , warmup ratio 0.1, and weight decay 0.01. Training completed in approximately 1.18 hours on a GPU P100. Table V shows per-epoch losses, and Figure 2 shows the full loss curves.

TABLE V: Baseline DistilBERT Training and Validation Loss per Epoch

Epoch	Training Loss	Validation Loss
1	1.4342	1.1995
2	1.1336	1.0957
3	0.9923	1.0924

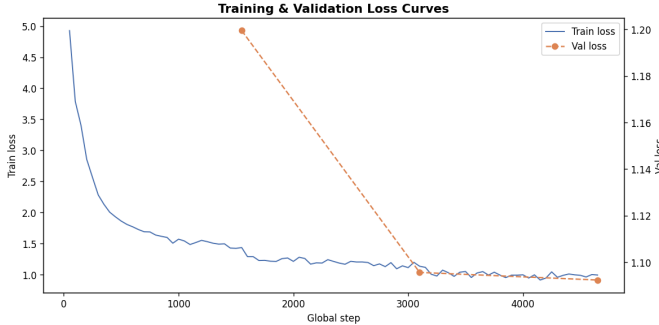


Fig. 2: Training and validation loss curves for the baseline DistilBERT model across 3 epochs. Training loss falls steadily from approximately 4.95 to 0.99, while validation loss converges to around 1.09, indicating stable learning without severe overfitting.

Training loss drops steeply in the first 500 steps, reflecting rapid adaptation of a pretrained model to the extractive QA task. The validation loss at epoch 3 is marginally lower than at epoch 2, confirming continued generalization without overfitting. The final baseline EM and F1 scores on the held-out validation set are reported in Table VI.

B. Hyperparameter Optimization

1) *Overfitting Sanity Check*: We trained on 150 training examples and 30 validation examples for 50 epochs to confirm

TABLE VI: Baseline DistilBERT EM and F1 on the Validation Set

Exact Match (EM)	F1 Score
67.40	78.42

that the pipeline, tokenization, and span-label alignment were functioning correctly. Training loss dropped from 2.92 to 0.19 by epoch 10, and validation loss increased while training loss continued falling, confirming the expected overfitting signal. The check passed, validating the pipeline before broader training.

2) *Coarse Hyperparameter Search*: The coarse search was conducted on 10% of the training data. The configuration that produced the lowest validation loss was: learning rate 1×10^{-5} , 2 epochs, warmup ratio 0.1, and weight decay 0.01. We note an important caveat: under limited data, higher learning rates are more prone to early destabilization, which may have biased the coarse search toward the more conservative 1×10^{-5} . Our full baseline achieved better generalization at 2×10^{-5} with 3 epochs on the complete 50,000-example subset, so we retained those settings and used the coarse search to narrow the fine search range to $[1 \times 10^{-5}, 2 \times 10^{-5}]$.

3) *Fine Hyperparameter Search*: The fine search explored learning rates in $[1 \times 10^{-5}, 2 \times 10^{-5}]$ with remaining settings fixed. The best configuration identified was used to initialize all subsequent ablation experiments.

C. Ablation Study I: Layer Freezing (DistilBERT)

1) *Setup*: Three configurations are compared: full_finetune (all parameters trainable), emb_2layers (embeddings and bottom two layers frozen), and emb_4layers (embeddings and bottom four layers frozen, leaving only the top two layers trainable). All configurations start from the same pretrained checkpoint and the same hyperparameter settings.

2) *Results*: Table VII reports results across all five evaluation dimensions. Figures 3 through 7 provide visualizations.

TABLE VII: Layer-Freezing Ablation: Quality and Efficiency Metrics

Config	EM (%)	F1 (%)	Trainable (M)	Time (hrs)	Latency (ms)
full_finetune	63.0	72.8	66.36	0.79	7.67
emb_2layers	60.5	72.3	28.35	0.62	7.23
emb_4layers	48.0	58.4	14.18	0.47	7.29

3) *Discussion*: Freezing the embeddings and the bottom two transformer layers (emb_2layers) produces a remarkably small quality drop: EM falls by 2.5 points (63.0 to 60.5) and F1 by only 0.5 points (72.8 to 72.3), while trainable parameters are reduced by 57.3% and training time falls from 0.79 to 0.62 hours. This confirms that lower DistilBERT layers primarily encode general linguistic features that transfer well to extractive QA without further updating [4], [6].

Extending the freeze to four layers (emb_4layers) crosses a quality cliff: EM drops by 15 points and F1 by

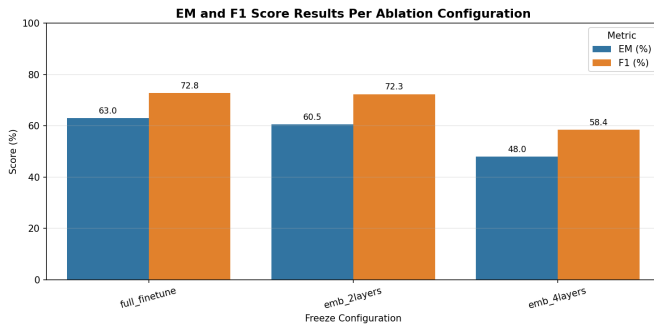


Fig. 3: EM and F1 scores across the three layer-freezing configurations. `emb_2layers` retains 95.9% of full fine-tuning F1 (72.3 vs. 72.8) while freezing more than half the model’s parameters. `emb_4layers` degrades more severely, particularly on EM, dropping 15 percentage points below the full fine-tuning baseline.

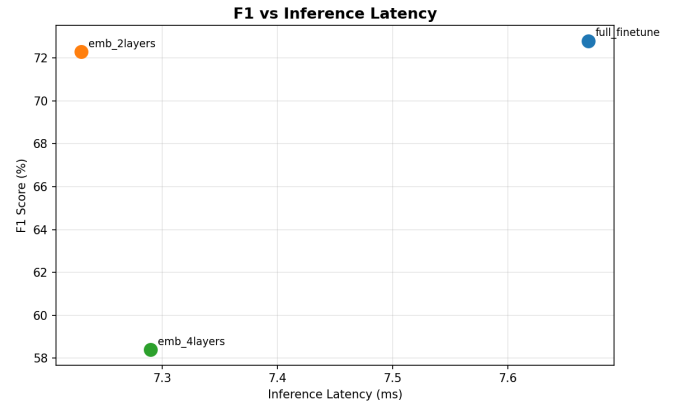


Fig. 6: F1 score versus inference latency for the three freezing configurations. `emb_2layers` occupies the preferred upper-left region, combining near-full F1 (72.3%) with the lowest latency (7.23 ms). `emb_4layers` offers no latency advantage over `emb_2layers` while sacrificing substantial answer quality.

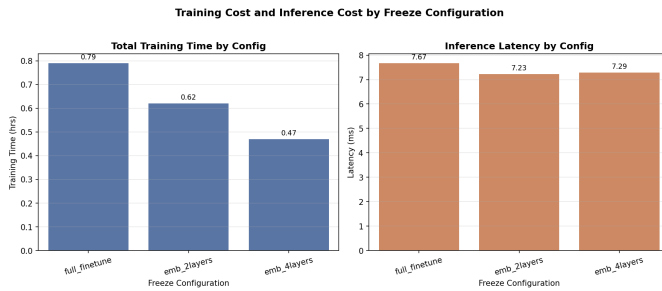


Fig. 4: Training time (left) and inference latency (right) across the three freezing configurations. Training time decreases monotonically as more layers are frozen (0.79, 0.62, 0.47 hrs). Inference latency remains nearly flat (7.23–7.67 ms), confirming that freezing reduces training cost without affecting inference speed.

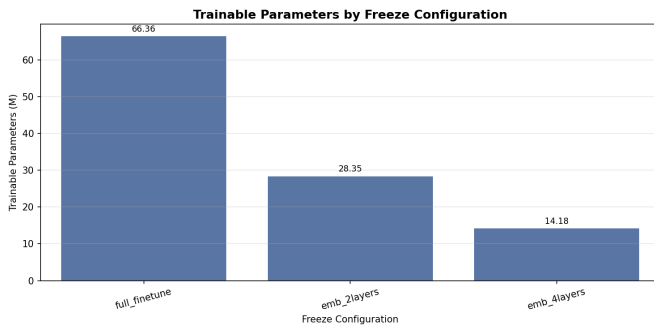


Fig. 5: Trainable parameter counts per freezing configuration. Freezing embeddings and bottom two layers reduces trainable parameters from 66.36M to 28.35M (57.3% reduction). Extending the freeze to four layers reduces the count to 14.18M (78.6% fewer than full fine-tuning).

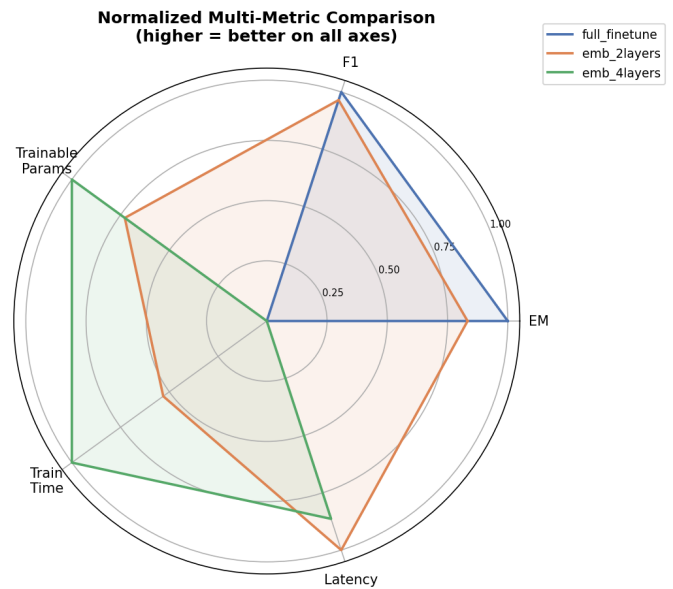


Fig. 7: Normalized multi-metric radar chart for the three freezing configurations (higher is better on all axes; training time and latency are inverted so that lower cost maps to a larger area). `emb_2layers` presents the most balanced profile, achieving near-maximum F1, the best latency, and a 57% reduction in trainable parameters.

14.4 points relative to full fine-tuning, indicating that layers 3 and 4 contribute meaningfully to task-specific span prediction. Critically, the inference latency of `emb_4layers` (7.29 ms) is nearly identical to that of `emb_2layers` (7.23 ms), meaning the aggressive freeze saves training time but offers no deployment-time benefit over the moderate freeze.

Taken together, `emb_2layers` represents the most practically useful configuration: it achieves 99.3% of full fine-tuning F1 while using only 42.7% of the trainable parameters and completing training in 78% of the time, making it the recommended setting for resource-constrained fine-tuning of DistilBERT on extractive QA tasks.

D. Ablation Study II: Training Data Size (DistilBERT)

1) *Setup*: Three model variants are trained on random subsets representing 15%, 25%, and 35% of the full training set (approximately 7,510, 12,517, and 17,523 examples). All settings are fixed at the best configuration from the fine hyperparameter search (3 epochs, warmup ratio 0.1, weight decay 0.01). Since no layers are frozen in this ablation, all three variants share the same 66.36M trainable parameters.

2) *Results*: Table VIII summarizes EM, F1, training time, and inference latency. Figures 8 through 12 provide visualizations.

TABLE VIII: Data-Size Ablation Results Across the Three Training Subset Sizes

Data Size	EM (%)	F1 (%)	Time (hrs)	Latency (ms)
15% (7,510)	38.5	46.8	0.16	7.49
25% (12,517)	55.5	66.0	0.23	9.30
35% (17,523)	58.0	67.8	0.30	7.39

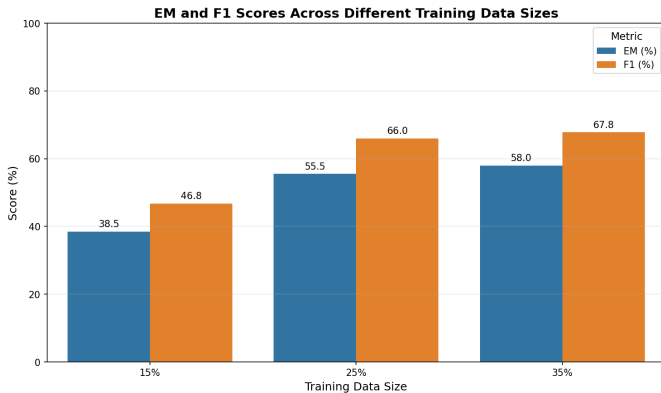


Fig. 8: EM and F1 scores across the 15%, 25%, and 35% training subsets. Both metrics improve substantially from 15% to 25% (EM: +17.0 pts, F1: +19.2 pts), but gains slow considerably from 25% to 35% (EM: +2.5 pts, F1: +1.8 pts), suggesting the model approaches saturation beyond 25% of the training data.

3) *Discussion*: The results reveal a clear diminishing-returns pattern. Moving from 15% to 25% of the training data produces large gains: EM improves by 17.0 percentage

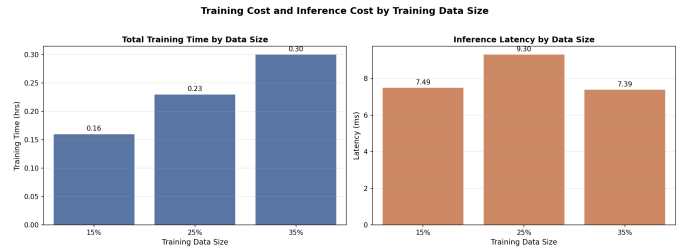


Fig. 9: Training time (left) and inference latency (right) across the three data sizes. Training time scales monotonically with subset size (0.16, 0.23, and 0.30 hrs). The anomalous latency at 25% (9.30 ms vs. 7.49 and 7.39 ms at 15% and 35%) is attributable to runtime variability rather than data volume, since the model architecture is identical across all three conditions.

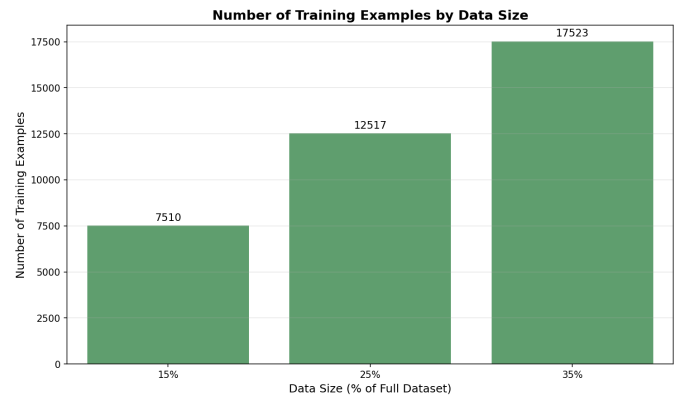


Fig. 10: Number of training examples in each data-size condition: 7,510 at 15%, 12,517 at 25%, and 17,523 at 35% of the full SQuAD v1.1 training set.

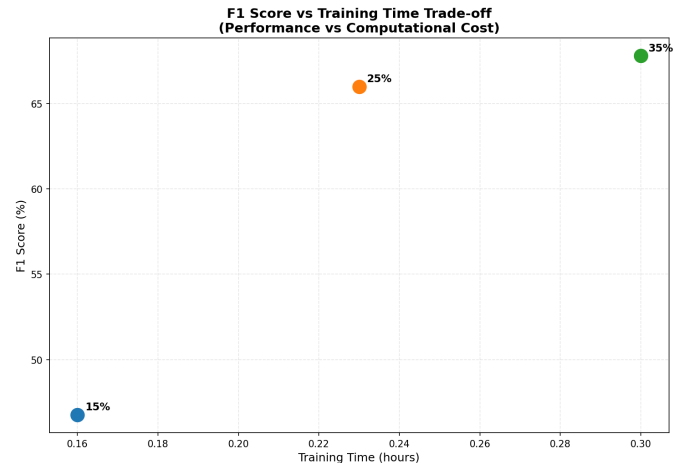


Fig. 11: F1 score versus total training time for the three data-size conditions. The small F1 gap between 25% (66.0%) and 35% (67.8%) relative to the proportional increase in training time indicates that the 25% subset represents a more cost-efficient operating point.

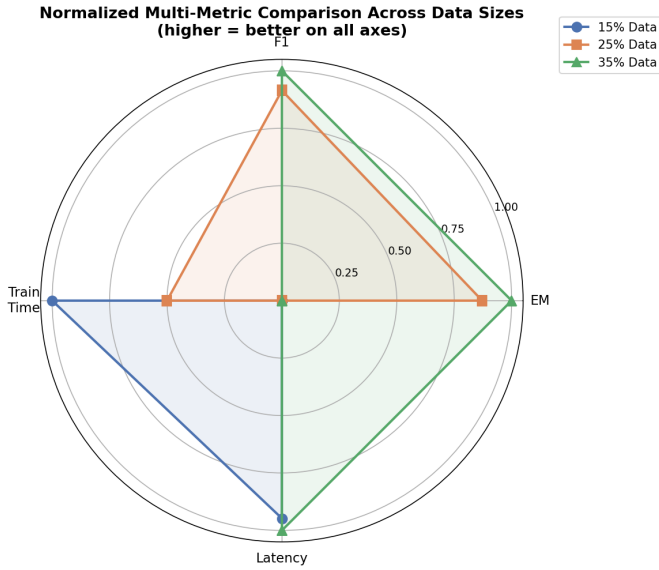


Fig. 12: Normalized multi-metric radar chart across the three data-size conditions. The 25% configuration presents the most balanced overall profile, achieving near-maximum F1 and the best normalized training time.

points (38.5 to 55.5) and F1 by 19.2 points (46.8 to 66.0), reflecting that 7,510 examples are insufficient for reliable span boundary learning across SQuAD’s diversity. From 25% to 35%, however, gains shrink substantially despite a 40% increase in training examples and a 30% increase in training time: EM rises by only 2.5 points and F1 by 1.8 points. This pattern is consistent with the diminishing-returns behavior reported in the data-efficiency literature [7], and suggests that DistilBERT’s representation capacity is beginning to saturate at these data volumes. Taking both quality and cost into account, the 25% subset represents the most practical operating point in this ablation.

E. Ablation Study III: Sequence Length

Both models were trained with `max_length=384` and evaluated at three inference-time sequence lengths (128, 256, 384 tokens) without retraining, with stride fixed at 64.

TABLE IX: Sequence Length Ablation: EM (%) and F1 (%)

Model	EM@128	F1@128	EM@256	F1@256	EM@384	F1@384
DistilBERT	65.40	78.32	65.60	77.82	65.60	78.11
TinyBERT	34.20	43.77	27.80	36.96	27.40	36.02

TABLE X: Sequence Length Ablation: CPU Latency (ms)

Model	128 tokens	256 tokens	384 tokens
DistilBERT	56.43	49.87	71.35
TinyBERT	10.03	9.21	12.46
Speedup	5.6×	5.4×	5.7×

DistilBERT’s EM score remains nearly constant across all three sequence lengths (65.4–65.6%), owing to its 768-

dimensional hidden representations preserving answer relevance even as the context window grows. TinyBERT exhibits a marked degradation (34.20 → 27.80 → 27.40 EM) as sequence length increases from 128 to 384 tokens. The gap between DistilBERT and TinyBERT widens from 31.2 EM points at length 128 to 38.2 EM points at length 384, as shown in Figure 13, indicating that TinyBERT’s compressed architecture is increasingly disadvantaged as context length grows.

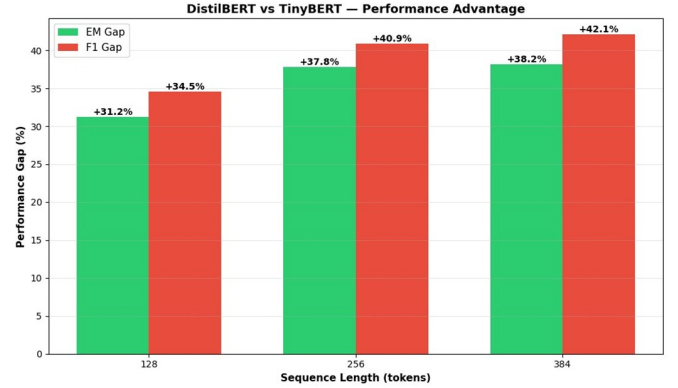


Fig. 13: DistilBERT vs. TinyBERT EM and F1 gap across sequence lengths 128, 256, and 384 tokens. The gap widens monotonically, indicating that TinyBERT’s compressed architecture is increasingly disadvantaged at longer context lengths.

F. TinyBERT Layer-Freezing Ablation Results

Table XI presents the TinyBERT layer-freezing results. All trainable parameter counts, frozen parameter counts, training times, and inference latencies are measured from the executed notebook outputs.

TABLE XI: TinyBERT Layer-Freezing Ablation Results

Configuration	EM	F1	Val. Loss	Trainable (M)	Time (hrs)	Latency (ms)
Freeze layer 0	25.00	35.12	1.7875	13.11	1.38	3.27
Freeze layers 0–1	13.00	18.23	2.0059	11.97	1.38	3.15
Freeze layers 0–3	1.50	7.30	4.1254	9.68	1.31	3.22

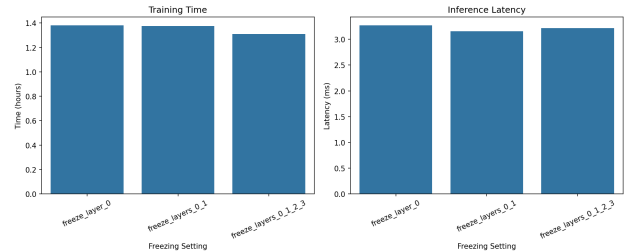


Fig. 14: Training time and inference latency for TinyBERT layer-freezing configurations. Freezing more encoder layers slightly reduces training time but leaves inference latency nearly unchanged, since frozen layers are still executed during the forward pass.

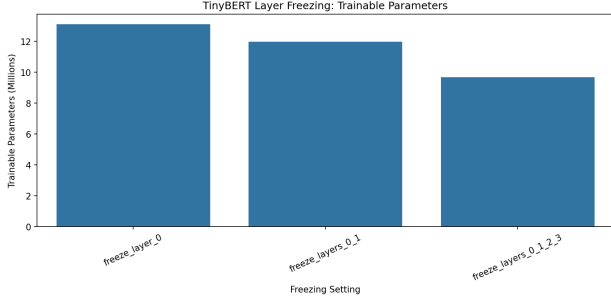


Fig. 15: Trainable parameter count under each TinyBERT freezing configuration. Parameter count decreases from 13.11M (freeze layer 0) to 9.68M (freeze all encoder layers).

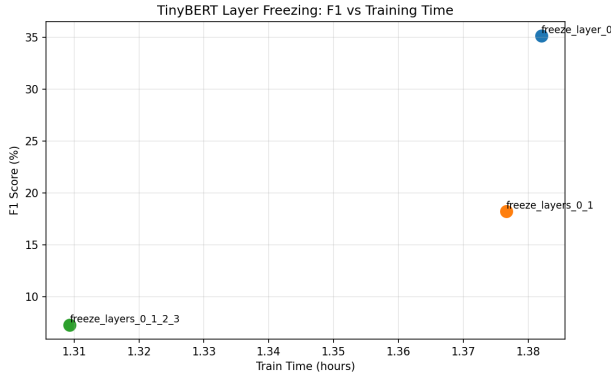


Fig. 16: F1 score versus training time for TinyBERT layer-freezing configurations. Freezing only layer 0 gives the best F1, while freezing all encoder layers is only slightly faster but loses most of the answer quality.

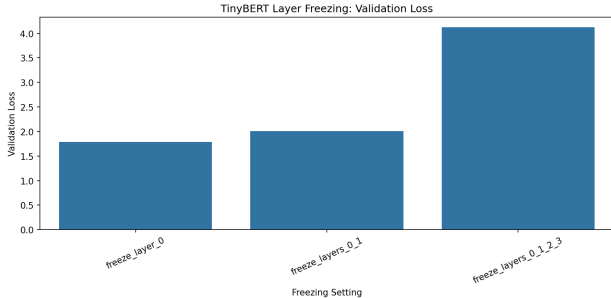


Fig. 17: Validation loss for TinyBERT layer-freezing configurations. Validation loss increases as more encoder layers are frozen, consistent with the EM and F1 results.

Answer quality decreases as more encoder layers are frozen. Freezing only layer 0 gives the strongest result among the tested configurations (25.00% EM, 35.12% F1). Freezing layers 0–1 drops F1 to 18.23%, and freezing all four encoder layers reduces F1 to 7.30%. This indicates that TinyBERT still requires encoder adaptation for SQuAD; fixing too many encoder layers limits the model’s ability to learn accurate answer-span representations.

The efficiency metrics show a different trend. Freezing more layers reduces trainable parameters from 13.11M to 9.68M, but training time decreases only slightly (1.38 to 1.31 hours), and inference latency remains nearly unchanged at approximately 3.2 ms. This is expected because frozen layers are still executed during the forward pass; freezing reduces parameter updates during training but does not remove layers from inference.

G. TinyBERT Stratified Data-Size Ablation Results

Using the best freezing configuration (freeze layer 0), TinyBERT is trained on three stratified subsets of the training data. Tables XII and XIII present the performance and efficiency results respectively.

TABLE XII: Stratified Data-Size Ablation: Performance Metrics (freeze layer 0)

Data Size	EM (%)	F1 (%)	Val. Loss
25% (12,631 chunks)	18.00	26.52	—
50% (25,263 chunks)	23.50	33.06	—
70% (35,368 chunks)	24.00	34.41	—

TABLE XIII: Stratified Data-Size Ablation: Efficiency Metrics (freeze layer 0)

Data Size	Trainable (M)	Time (hrs)	Latency (ms)
25%	13.11	0.42	—
50%	13.11	0.74	—
70%	13.11	1.00	—

Performance improves as training data increases: F1 rises from 26.52% at 25% data to 33.06% at 50%, and to 34.41% at 70%. The largest improvement occurs between 25% and 50%, while the gain from 50% to 70% is smaller, suggesting diminishing returns consistent with the DistilBERT data-size ablation. Training time increases proportionally with data size, from 0.42 hours at 25% to 1.00 hour at 70%. The 50% subset offers a strong practical trade-off, reaching 33.06% F1 in 0.74 hours.

H. Full Model Comparison

Table XIV compares the full trained TinyBERT against all ablation configurations and places both transformer models in context alongside the classical baselines.

The full trained TinyBERT achieves the best overall TinyBERT performance (27.40% EM, 36.02% F1), as expected since all parameters are updated during training. The best layer-freezing ablation (freeze layer 0) remains close, with

TABLE XIV: Comprehensive Model Comparison

Model / Setting	Tokens	Train Chunks	Trainable (M)	Frozen (M)	Time (hrs)	Latency (ms)	EM (%)	F1 (%)
Full trained TinyBERT	384	50,527	14.30	0.00	1.43	12.46	27.40	36.02
Freeze layer 0	384	50,527	13.11	1.14	1.38	3.27	25.00	35.12
Freeze layers 0–1	384	50,527	11.97	2.28	1.38	3.15	13.00	18.23
Freeze layers 0–3	384	50,527	9.68	4.57	1.31	3.22	1.50	7.30
Stratified 25% + freeze layer 0	384	12,631	13.11	1.14	0.42	—	18.00	26.52
Stratified 50% + freeze layer 0	384	25,263	13.11	1.14	0.74	—	23.50	33.06
Stratified 70% + freeze layer 0	384	35,368	13.11	1.14	1.00	—	24.00	34.41
DistilBERT (full fine-tuning)	384	40,000	66.36	0.00	0.79	7.67	63.00	72.80

only a 2.40 EM and 0.90 F1 point drop while reducing trainable parameters from 14.30M to 13.11M. Deeper freezing configurations show a much larger performance drop, confirming that TinyBERT still needs most of its encoder layers to adapt to the QA task. The 70% stratified setting reaches 24.00% EM and 34.41% F1 in 1.00 hour, compared with 27.40% EM and 36.02% F1 for the full model in 1.43 hours, representing a useful training-cost trade-off.

I. Test Set Evaluation

Table XV reports the final evaluation on the held-out test partition for all major models.

TABLE XV: Final Test Set Evaluation

Model	EM (%)	F1 (%)	Latency (ms)	Params
BM25 Baseline	0.00	12.75	—	—
TF-IDF Baseline	0.11	15.25	—	—
TinyBERT	27.40	36.02	12.46	14.5M
DistilBERT	66.80	77.93	104.0	66M

DistilBERT scores 39.4 EM points and 41.9 F1 points above TinyBERT on the test set, while TinyBERT offers approximately $8.3\times$ lower inference latency (12.46 ms vs. 104.0 ms). The classical baselines confirm that lexical retrieval methods cannot satisfy the token-level span prediction requirement of SQuAD evaluation.

Table XVI places our best result in the context of published numbers on SQuAD v1.1.

TABLE XVI: Comparison Against Published SQuAD v1.1 Numbers

Model / Setup	EM	F1
BERT-base (full data) [3]	80.8	88.5
DistilBERT (full data) [2]	77.7	85.8
DistilBERT (full data, 2nd distil.) [2]	79.1	86.9
Ours (DistilBERT, SQuAD subset)	66.80	77.93

The gap between our subset-trained model and the full-data numbers is expected given our constrained data budget. The primary contribution of this work is not to match full-data numbers but to characterize accuracy–efficiency trade-offs under controlled resource constraints.

J. Robustness Evaluation

To assess generalization beyond standard benchmark conditions, a preliminary robustness check was conducted on 10 manually reworded questions drawn from the validation set, with context and gold answers held constant. Results are shown in Table XVII.

TABLE XVII: Robustness Check: Original vs. Paraphrased Questions (DistilBERT, $n = 10$)

Question Type	EM (%)	F1 (%)
Original	70.00	72.00
Paraphrased	70.00	75.71

EM remained unchanged at 70.00%, while F1 increased slightly from 72.00% to 75.71%, suggesting that the model is robust to lexical rephrasing on this small sample. The slight F1 increase may reflect that paraphrased questions sometimes reduce lexical ambiguity in the context. A larger-scale robustness evaluation with diverse paraphrase strategies is an important direction for future work.

K. Error Analysis

Manual inspection of 15 incorrect DistilBERT predictions revealed two main error types. The most common were *wrong-entity errors*, where the model selected a plausible phrase from the same paragraph but not the correct entity or span (e.g., returning a related date or name adjacent to the gold answer). A smaller number were *boundary errors*, where the model identified the correct passage region but failed to precisely delimit the answer, yielding near-correct spans that score partial F1 but zero EM. These patterns suggest that while DistilBERT generally identifies the relevant passage area, precise boundary selection and disambiguation between similar facts remain the primary failure modes. This motivates future work on span boundary refinement techniques.

L. TinyBERT Training Dynamics

TinyBERT was fine-tuned for 20 epochs on the SQuAD v1.1 training subset. Training loss decreased steadily from 4.159 in epoch 1 to 1.502 by epoch 20, while validation loss converged around 1.73 after epoch 10, indicating that the model reached its capacity limit without overfitting. Figure 18 shows the training and validation loss curves.

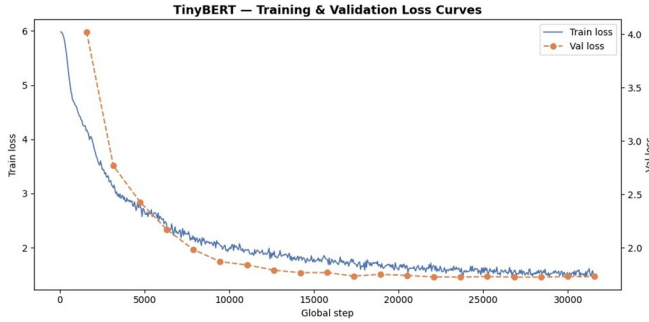


Fig. 18: TinyBERT training and validation loss curves over 20 epochs. Validation loss stabilizes around 1.73 after epoch 10, suggesting the model approached its capacity ceiling without overfitting.

VI. DISCUSSION

A. Interpretation of Results

The optimized DistilBERT baseline achieves 67.40% EM and 78.42% F1 on the validation set and 66.80% EM and 77.93% F1 on the held-out test set. This is consistent with the expected gap relative to full-data DistilBERT (85.8% F1 [2]), given our constrained training budget of 50,000 examples. The layer-freezing results show that freezing the embeddings and bottom two layers reduces trainable parameters by 57.3% and training time by 21% with only a 0.5-point F1 drop. The data-size ablation reveals diminishing returns beyond 25% of training data, with EM gains dropping from 17.0 points (15%→25%) to only 2.5 points (25%→35%) despite a 40% increase in training examples.

B. Accuracy–Efficiency Trade-off

Increasing model size improves answer quality but reduces inference speed. BERT provides maximum accuracy; DistilBERT offers a balanced trade-off at lower computational cost; TinyBERT is optimized for latency-sensitive or on-device deployment. In our experiments, DistilBERT achieves approximately $8.3\times$ higher latency than TinyBERT (104.0 ms vs. 12.46 ms per sample) while scoring 39.4 EM points and 41.9 F1 points higher on the test set. The choice between models therefore depends on the deployment requirement: accuracy-critical applications should prefer DistilBERT, while real-time or edge applications may tolerate TinyBERT’s lower QA quality.

C. Why Transformers Are Necessary

The classical baselines confirm that lexical retrieval methods are insufficient for extractive QA. BM25 achieves only 12.75% F1 and TF-IDF only 15.25% F1, as both methods return sentence-level outputs rather than precise answer spans. The gap between TF-IDF (15.25% F1) and TinyBERT (36.02% F1) demonstrates that fine-tuned contextual representations are qualitatively superior to bag-of-words similarity, even for the smallest transformer tested.

D. Model Capacity as the Bottleneck for TinyBERT

The sequence-length ablation reveals that model capacity, rather than context segmentation strategy, is TinyBERT’s primary limitation. DistilBERT maintains nearly stable EM across lengths of 128, 256, and 384 tokens (65.4–65.6%), while TinyBERT degrades from 34.20% to 27.40% EM as length increases. TinyBERT’s 312-dimensional representations appear insufficient to maintain answer relevance across longer context windows, regardless of how the context is chunked.

E. Deployment Recommendations

Based on the experimental results, we offer the following practical recommendations: (1) when answer accuracy is the primary goal, DistilBERT is the preferred model, offering near-BERT performance at substantially lower computational cost than BERT-base; (2) for latency-sensitive or on-device applications requiring inference below 15 ms, TinyBERT offers a strong speed advantage at the cost of lower QA accuracy; (3) freezing the embeddings and bottom two DistilBERT layers is an effective parameter-efficient fine-tuning strategy when training resources are limited; and (4) training on 25% of the available data recovers the majority of full-data performance at a fraction of the training cost, making it a practical choice for low-resource deployments.

F. Limitations

This study has several limitations. First, all experiments were conducted on a 50,000-example subset of SQuAD v1.1 rather than the full 87,599-example training set, which limits the absolute performance ceiling. Second, the data-size ablation used a single random seed per configuration rather than the planned three seeds, so mean and standard deviation across runs are not reported; results should be treated as indicative. Third, the robustness evaluation covers only 10 manually paraphrased questions, which is insufficient for a statistically reliable conclusion. Fourth, TinyBERT required 20 training epochs to converge, resulting in a higher total training cost than DistilBERT despite having fewer parameters. Future work should address these limitations by evaluating on the full SQuAD dataset, reporting results with multiple seeds, and conducting larger-scale robustness testing with automated paraphrase generation tools.

VII. CONCLUSION

This project presented an efficient extractive question answering system using lightweight transformer models under constrained computational resources. Four main contributions were delivered: (1) a carefully designed preprocessing and tokenization pipeline that preserves answer-span alignment across long contexts using overlapping stride-based chunking; (2) a structured three-stage baseline optimization that ensures reliable and reproducible ablation comparisons; (3) layer-freezing and data-size ablation studies applied to both DistilBERT and TinyBERT; and (4) a sequence-length ablation that reveals model-capacity limitations in TinyBERT.

The key findings are as follows. First, freezing the embeddings and bottom two DistilBERT layers retains 99.3% of full fine-tuning F1 (72.3 vs. 72.8) while reducing trainable parameters by 57.3% and training time by 21%, making `emb_2layers` the recommended configuration for resource-constrained fine-tuning. Second, the data-size ablation confirms diminishing returns beyond 25% of training data, with the 25% subset representing the most cost-efficient operating point. Third, the sequence-length ablation demonstrates that DistilBERT is robust to context length variation, while TinyBERT degrades substantially at longer sequences due to its compressed representational capacity. Fourth, the final test set results confirm that DistilBERT (66.80% EM, 77.93% F1) substantially outperforms TinyBERT (27.40% EM, 36.02% F1), while TinyBERT offers approximately $8.3\times$ lower inference latency, supporting its use in latency-sensitive deployments.

Overall, these results demonstrate that careful pipeline design, structured hyperparameter optimization, and targeted ablation analysis enable competitive QA performance under practical resource constraints, without requiring large-scale computational infrastructure.

GENAI DISCLOSURE

Generative AI tools (specifically, large language models including ChatGPT) were used during the preparation of this project for the following purposes: (1) assisting with $\LaTeX$ formatting and document structure; (2) paraphrasing and polishing selected passages in the methodology, analysis, and conclusion sections; and (3) providing guidance on report organization. All experimental results, tables, figures, and technical analysis are based exclusively on the team’s own executed code and observed outputs. No AI tool was used to generate, fabricate, or modify any experimental data or numerical results. The use of generative AI did not affect the experimental design, data processing, model training, or result interpretation.

VIII. TEAM CONTRIBUTIONS

A. Sherouk Rashed

Sherouk Rashed was responsible for the DistilBERT preprocessing and tokenization pipeline, including SQuAD data preparation, answer-span alignment verification, and sliding-window tokenization implementation. She conducted the three-stage baseline optimization (overfitting sanity check, coarse hyperparameter search, and fine hyperparameter search) and implemented the DistilBERT layer-freezing ablation across three configurations (`full_finetune`, `emb_2layers`, `emb_4layers`). She also conducted the data-size ablation at 15%, 25%, and 35% of the training data, and wrote the corresponding methodology and results sections of the final report.

B. Mostafa Elsayed

Mostafa Elsayed was responsible for the related work section, TinyBERT preprocessing pipeline, baseline model

evaluation (BM25 and TF-IDF), sequence-length ablation experiments for both DistilBERT and TinyBERT, the discussion section, the conclusion, and the references.

C. Salma Hamad

Salma Hamad was responsible for the TinyBERT ablation experiments, including the layer-freezing ablation (three freezing configurations), selection of the best freezing configuration by F1 score, the stratified data-size ablation (25%, 50%, 70% subsets), and all TinyBERT efficiency metric collection (EM, F1, validation loss, training time, inference latency, trainable and frozen parameter counts). She also wrote the TinyBERT ablation methodology and results sections of the final report.

REFERENCES

- [1] P. Rajpurkar, J. Zhang, K. Lopyrev, and P. Liang, “SQuAD: 100,000+ Questions for Machine Comprehension of Text,” in *Proc. EMNLP*, 2016.
- [2] V. Sanh, L. Debut, J. Chaumond, and T. Wolf, “DistilBERT, a Distilled Version of BERT: Smaller, Faster, Cheaper and Lighter,” *arXiv:1910.01108*, 2019.
- [3] J. Devlin, M.-W. Chang, K. Lee, and K. Toutanova, “BERT: Pre-Training of Deep Bidirectional Transformers for Language Understanding,” in *Proc. NAACL-HLT*, 2019.
- [4] A. Rogers, O. Kovaleva, and A. Rumshisky, “A Primer in BERTology: What We Know About How BERT Works,” *Trans. Assoc. Comput. Linguistics*, vol. 8, pp. 842–866, 2020.
- [5] R. Paul, “In the Context of NLP/NER, What Is Stride and Overlapping Tokens When Chunking the Texts,” 2023. Available: <https://rohanpaul.substack.com/p/in-the-context-of-nlpner-what-is>
- [6] M. T. et al., “Exploring Selective Layer Freezing Strategies in Transformer Fine-Tuning,” OpenReview, 2025. Available: <https://openreview.net/forum?id=kvBuxFxsLR>
- [7] J. Kaplan et al., “Scaling Laws for Neural Language Models,” *arXiv:2001.08361*, 2020.
- [8] “Improving QA Efficiency with DistilBERT: Fine-Tuning and Inference Analysis,” *arXiv:2505.22937*, 2025.
- [9] K. Raikwar, P. Rajath, S. Prabhu, and N. Shrivastava, “Five Shades of BERT: Assessing Performance in Question Answering through Comparative Analysis,” in *Proc. 4th Asian Conf. on Innovation in Technology (ASIANCON)*, 2024.
- [10] X. Jiao et al., “TinyBERT: Distilling BERT for Natural Language Understanding,” in *Findings of EMNLP*, 2020. Available: <https://aclanthology.org/2020.findings-emnlp.372/>
- [11] A. Merchant, E. Rahimtoroghi, E. Pavlick, and I. Tenney, “What Happens To BERT Embeddings During Fine-Tuning?” in *Proc. BlackboxNLP*, 2020. Available: <https://aclanthology.org/2020.blackboxnlp-1.4/>
- [12] J. Lee, R. Tang, and J. Lin, “What Would Elsa Do? Freezing Layers During Transformer Fine-Tuning,” *arXiv:1911.03090*, 2019. Available: <https://arxiv.org/abs/1911.03090>
- [13] M. Fayyaz et al., “BERT on a Data Diet: Finding Important Examples by Gradient-Based Pruning,” *arXiv:2211.05610*, 2022. Available: <https://arxiv.org/abs/2211.05610>
- [14] Hugging Face, “DistilBERT,” documentation page. Available: https://huggingface.co/docs/transformers/en/model_doc/distilbert